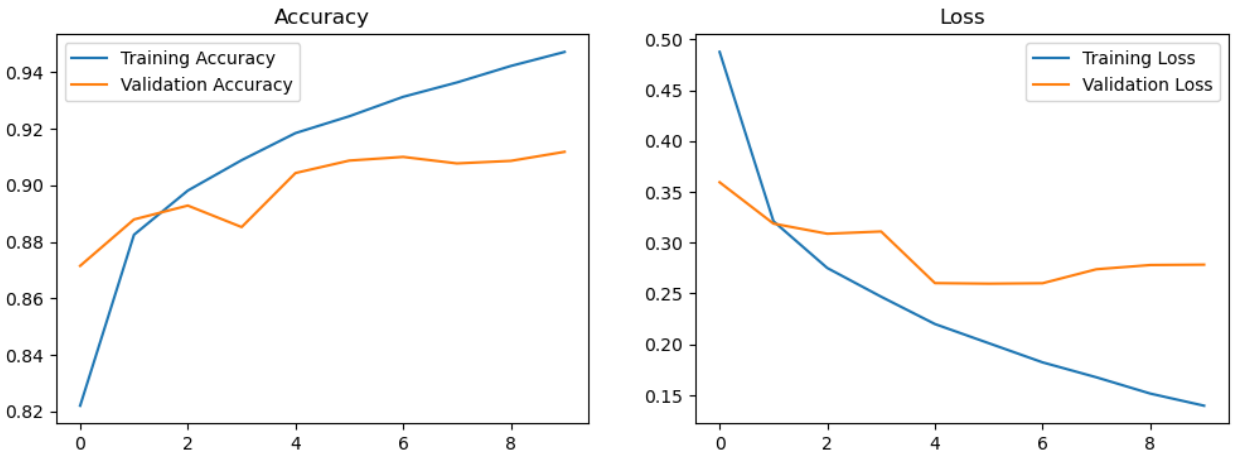
```
# Fashion MNIST Classification using CNN

import numpy as np

import tensorflow as tf

from tensorflow.keras import datasets, layers, models

import matplotlib.pyplot as plt


print("TensorFlow version:", tf.__version__)

# Load and preprocess the dataset

print("Loading Fashion MNIST dataset...")

(train_images, train_labels), (test_images, test_labels) =
datasets.fashion_mnist.load_data()


# Normalize pixel values to be between 0 and 1

train_images, test_images = train_images / 255.0, test_images / 255.0


# Reshape images to include channel dimension (28, 28, 1)

train_images = train_images.reshape((60000, 28, 28, 1))

test_images = test_images.reshape((10000, 28, 28, 1))


class_names = ['T-shirt/top', 'Trouser', 'Pullover', 'Dress', 'Coat',

        'Sandal', 'Shirt', 'Sneaker', 'Bag', 'Ankle boot']


print(f"Training set: {train_images.shape}")

print(f"Test set: {test_images.shape}")

# Visualize some samples

plt.figure(figsize=(10,10))
```

```
for i in range(25):
    plt.subplot(5,5,i+1)
    plt.xticks([])
    plt.yticks([])
    plt.grid(False)
    plt.imshow(train_images[i].reshape(28,28), cmap=plt.cm.binary)
    plt.xlabel(class_names[train_labels[i]])
plt.show()
```


```
# Build the CNN architecture
model = models.Sequential([
    layers.Conv2D(32, (3, 3), activation='relu', input_shape=(28, 28, 1)),
    layers.MaxPooling2D((2, 2)),
    layers.Conv2D(64, (3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),
```

```
    layers.Conv2D(64, (3, 3), activation='relu'),
    layers.Flatten(),
    layers.Dense(64, activation='relu'),
    layers.Dense(10, activation='softmax')
])

model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])

model.summary()
# Train the model
history = model.fit(train_images, train_labels, epochs=10,
                    validation_data=(test_images, test_labels))
# Plot accuracy and loss
plt.figure(figsize=(12, 4))
plt.subplot(1, 2, 1)
plt.plot(history.history['accuracy'], label='Training Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.title('Accuracy')
plt.legend()

plt.subplot(1, 2, 2)
plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.title('Loss')
```

```
plt.legend()

plt.show()
```




```
# Make predictions and visualize results

predictions = model.predict(test_images)


def plot_image(i, predictions_array, true_label, img):

  true_label, img = true_label[i], img[i].reshape(28,28)

  plt.grid(False)

  plt.xticks([])

  plt.yticks([])

  plt.imshow(img, cmap=plt.cm.binary)

  predicted_label = np.argmax(predictions_array)

  color = 'blue' if predicted_label == true_label else 'red'

  plt.xlabel(f"{class_names[predicted_label]} {100*np.max(predictions_array):2.0f}%
({class_names[true_label]})", color=color)


num_rows = 5

num_cols = 3

num_images = num_rows*num_cols
```

```
plt.figure(figsize=(2*2*num_cols, 2*num_rows))
for i in range(num_images):
   plt.subplot(num_rows, 2*num_cols, 2*i+1)
   plot_image(i, predictions[i], test_labels, test_images)
plt.show()
```